

Tree Borrows

Neven Villani,¹ Johannes Hostert,² Derek Dreyer,³ Ralf Jung²

PLDI'25

2025-06-21

¹Univ. Grenoble Alpes, Verimag

²ETH Zurich

³MPI-SWS

A typical optimization

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
  
    *x = 13;  
  
    *y = 20;  
  
    *x  
  
}
```

A typical optimization

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
  
    *x = 13;  
  
    *y = 20;  
  
    *x  
  
}
```

mutable thus disjoint

*x is unchanged

A typical optimization

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

mutable thus disjoint

*x has known value

*x is unchanged

always returns 13

A typical optimization

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
  
    *x = 13;  
  
    *y = 20;  
  
    13  
  
}
```

Type-level guarantees for references



`&mut` → mutation, no aliasing

`&` → aliasing, no mutation

Escape hatch: **unsafe**

```
unsafe {  
    // Code within this block has relaxed typechecking  
    . . .  
}
```

- can **bypass typechecks**
- necessary for **low-level manipulations**
- within **unsafe** it is **the programmer's responsibility** to check
 - that pointers are non-null
 - that memory is initialized
 - ...

What if **unsafe** code violates an invariant required for optimizations ?

Escape hatch: **unsafe**

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

```
fn main() {  
    let mut root = 42;  
    let ptr = &raw mut root;  
    let x = unsafe { &mut *ptr };  
    let y = unsafe { &mut *ptr };  
    println!("{}", write_both(x, y));  
}
```


Escape hatch: **unsafe**

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

```
fn main() {  
    let mut root = 42;  
    let ptr = &raw mut root;  
    let x = unsafe { &mut *ptr };  
    let y = unsafe { &mut *ptr };  
    println!("{}", write_both(x, y));  
}
```

Escape hatch: **unsafe**

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

```
fn main() {  
    let mut root = 42;  
    let ptr = &raw mut root;  
    let x = unsafe { &mut *ptr };  
    let y = unsafe { &mut *ptr };  
    println!("{}", write_both(x, y));  
}
```

The optimization is valid, it's the code that's wrong

- within **unsafe** it is **the programmer's responsibility** to check
 - that pointers are non-null
 - that memory is initialized
 - ...
 - compliance with Tree Borrows ^{NEW!}

→ new **proof obligations** on **unsafe** blocks

→ violation of these rules results in **Undefined Behavior**

The optimization is valid, it's the code that's wrong

- within `unsafe` it is the programmer's responsibility to check
 - that pointers are non-null
 - that memory is initialized
 - ...

Sounds familiar?

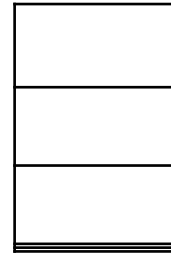
Stacked Borrows has the same purpose,
Tree Borrows is its successor.

- new proof obligations on `unsafe` blocks
- violation of these rules results in **Undefined Behavior**

Stacked Borrows

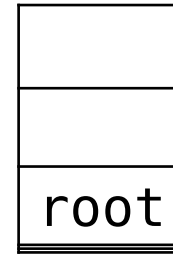
A **stack** is a natural way to track reborrows because mutable references are **well-bracketed**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
let val = write_both(x, y);
```



A **stack** is a natural way to track reborrows because mutable references are **well-bracketed**.

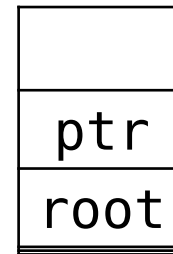
```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
let val = write_both(x, y);
```



- new stack at root

A **stack** is a natural way to track reborrows because mutable references are **well-bracketed**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
let val = write_both(x, y);
```



- pop until root
- push ptr

A **stack** is a natural way to track reborrows because mutable references are **well-bracketed**.

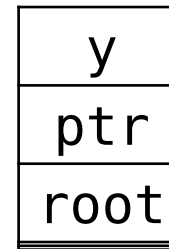
```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
let val = write_both(x, y);
```

x
ptr
root

- pop until ptr
- push x

A **stack** is a natural way to track reborrows because mutable references are **well-bracketed**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
let val = write_both(x, y);
```



- pop until ptr
- push y

A **stack** is a natural way to track reborrows because mutable references are **well-bracketed**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
let val = write_both(x, y);
```

y
ptr
root

- search for x
- search for y

A **stack** is a natural way to track reborrows because mutable references are **well-bracketed**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
let val = write_both(x, y);
```

y
ptr
root

UB: cannot use x when
it is not in the stack

SB was **implemented** in Miri

→ included in many projects' CI

→ several bugs detected (e.g. in stdlib)

SB was **implemented** in Miri

→ included in many projects' CI

→ several bugs detected (e.g. in stdlib)

However...

- references have static range
- ignores two-phased borrows
- prohibits reordering reads

SB was **implemented** in Miri

→ included in many projects' CI

→ several bugs detected (e.g. in stdlib)

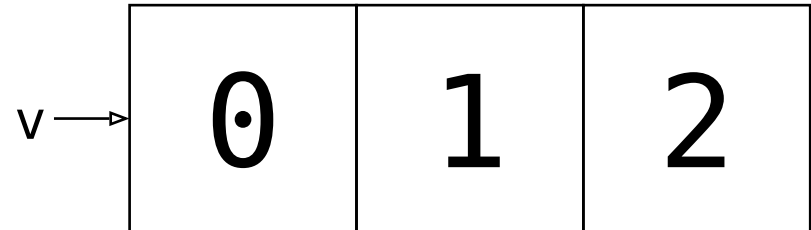
However...

- **references have static range**
- ignores two-phased borrows
- prohibits reordering reads

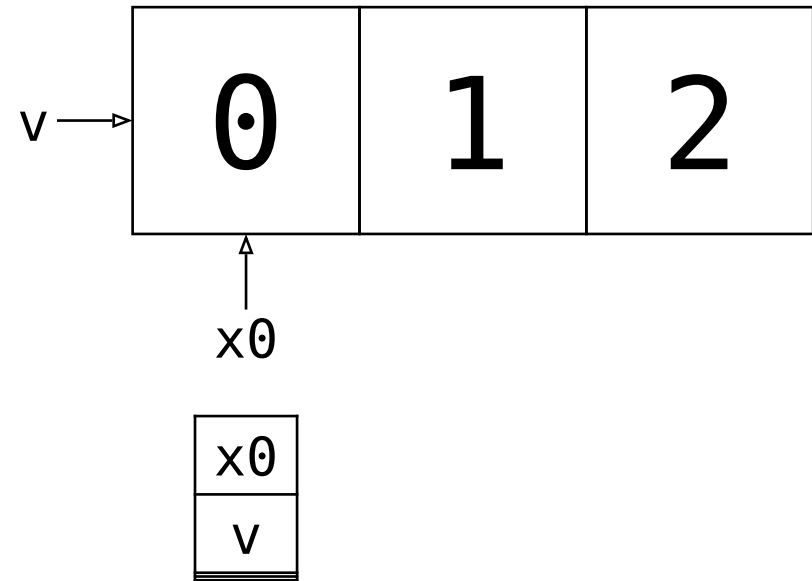
From Stacks to Trees


```
let mut v = vec![0, 1, 2];  
let x0 = &raw mut v[0];  
let x2 = &raw mut v[2];  
...
```

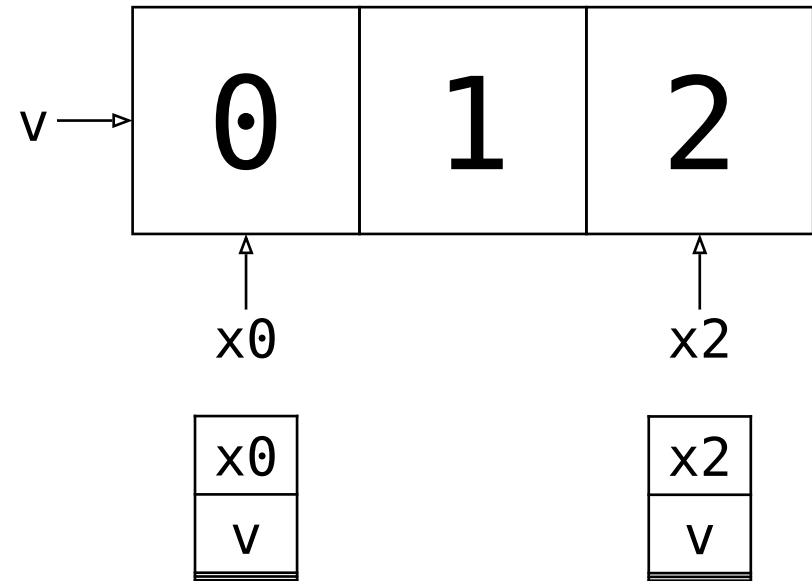
```
let mut v = vec![0, 1, 2];  
let x0 = &raw mut v[0];  
let x2 = &raw mut v[2];  
...
```



```
let mut v = vec![0, 1, 2];  
let x0 = &raw mut v[0];  
let x2 = &raw mut v[2];  
...
```



```
let mut v = vec![0, 1, 2];  
let x0 = &raw mut v[0];  
let x2 = &raw mut v[2];  
...
```

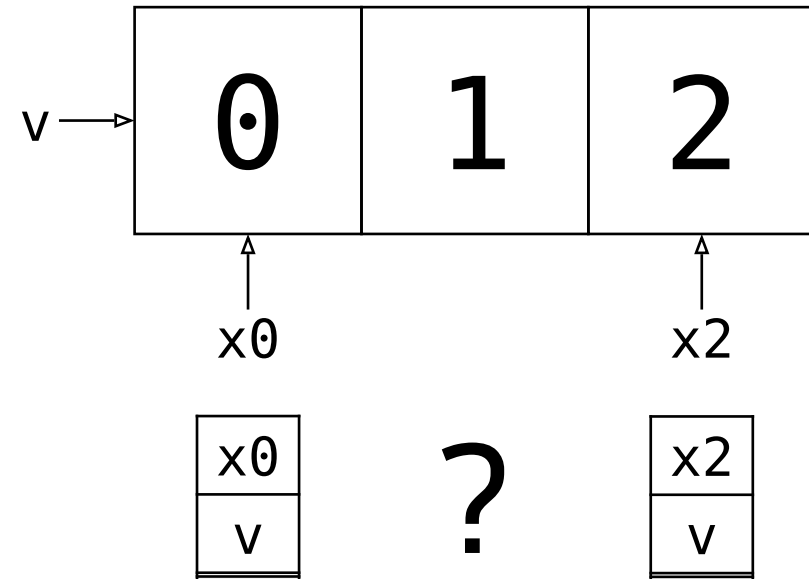
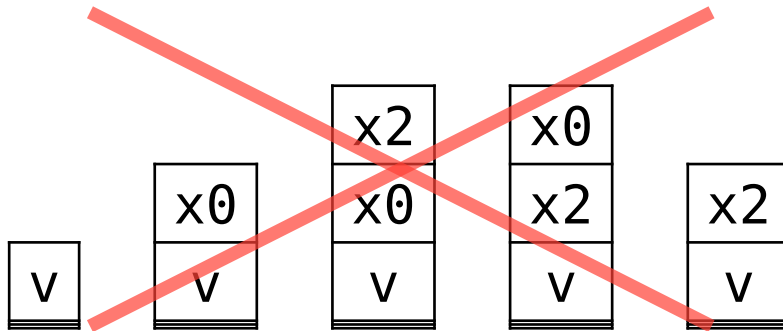


```

let mut v = vec![0, 1, 2];
let x0 = &raw mut v[0];
let x2 = &raw mut v[2];
...

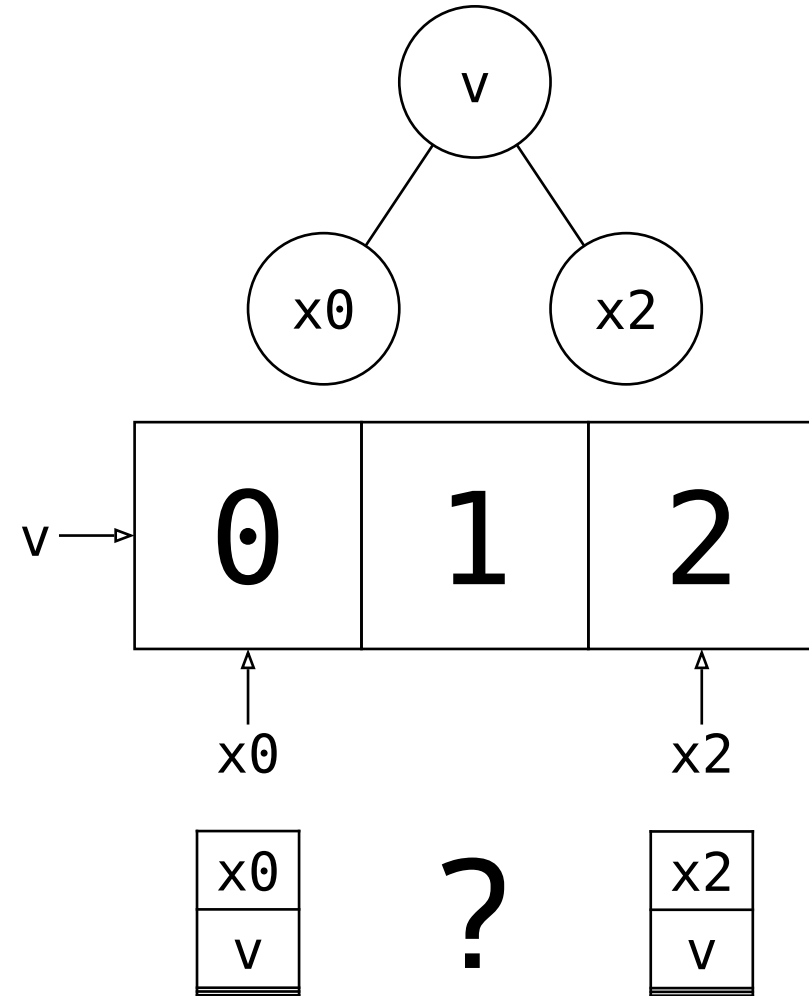
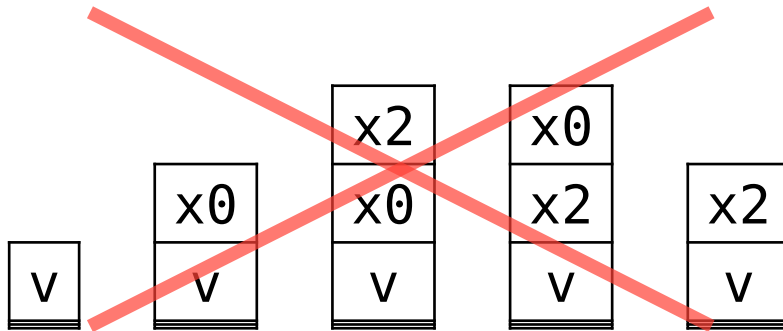
```

What stack at offset 1 ?



```
let mut v = vec![0, 1, 2];
let x0 = &raw mut v[0];
let x2 = &raw mut v[2];
...
```

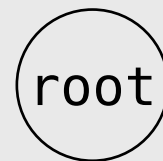
What stack at offset 1 ?



```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```

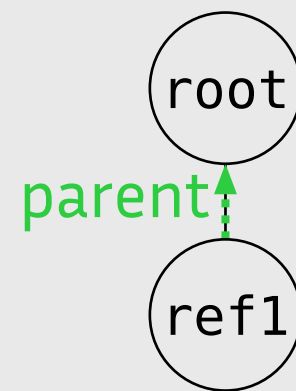
Relative positions in the tree

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```



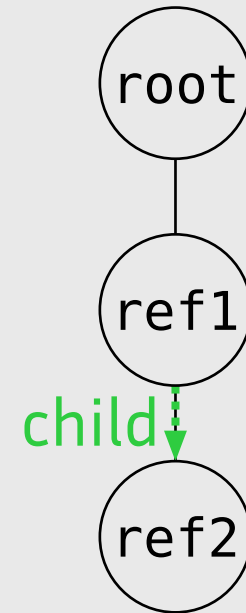
Relative positions in the tree

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```



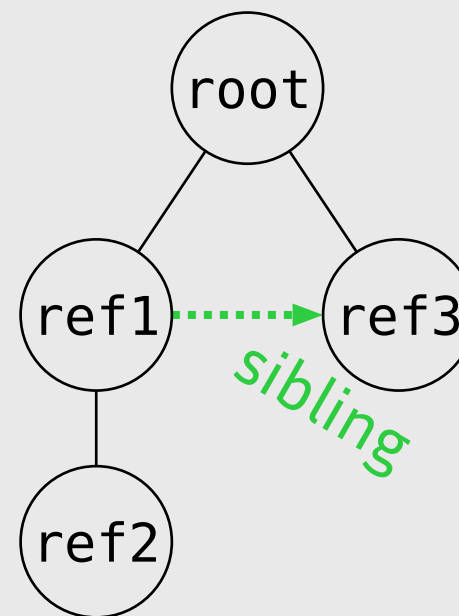
Relative positions in the tree

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```



Relative positions in the tree

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```

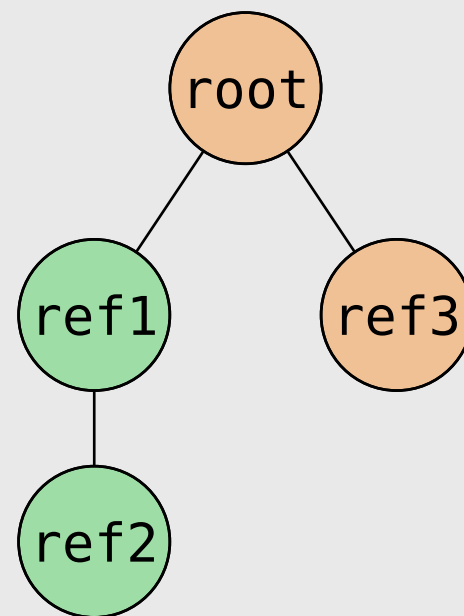


Relative positions in the tree

From Stacks to Trees

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```

foreign accesses



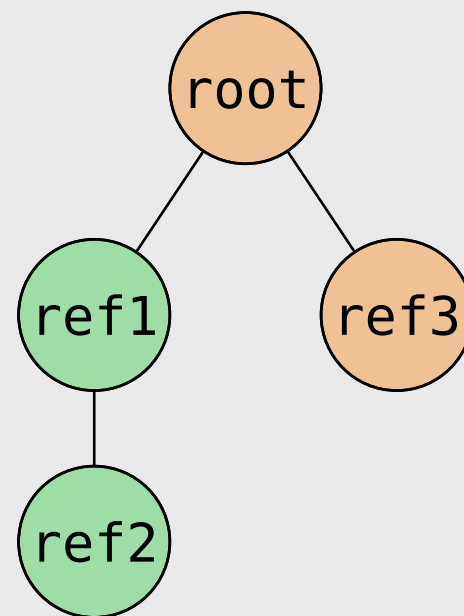
local accesses

Relative positions in the tree

From Stacks to Trees

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;  
  
let val = *ref2;
```

foreign accesses



local accesses

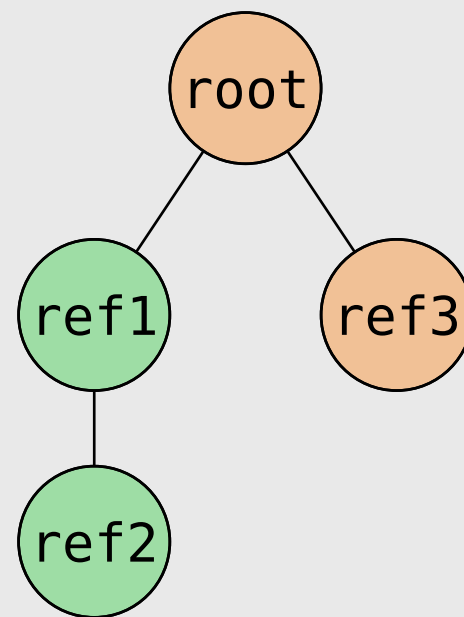
Relative positions in the tree

From Stacks to Trees

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```

```
let val = *ref2;  
let val = *ref3;
```

foreign accesses



local accesses

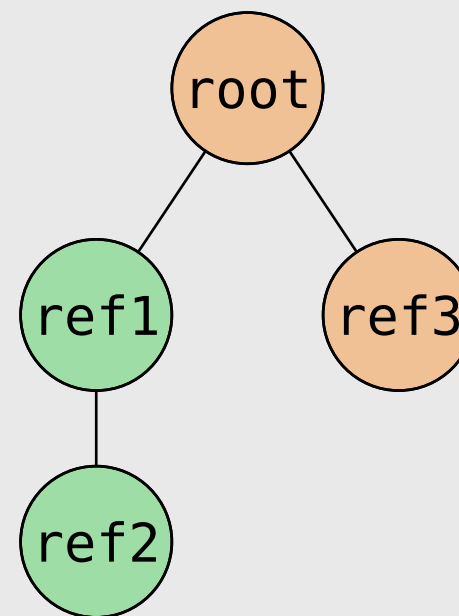
Relative positions in the tree

From Stacks to Trees

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```

```
let val = *ref2;  
let val = *ref3;  
*ref1 = 36;
```

foreign accesses



local accesses

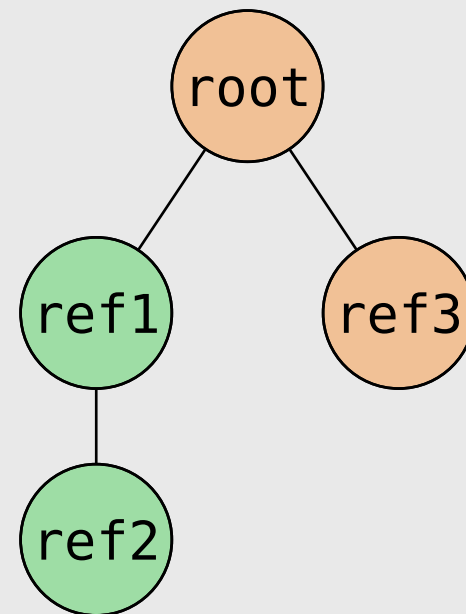
Relative positions in the tree

From Stacks to Trees

```
let mut root = 42;  
let ref1 = &mut root;  
let ref2 = &mut *ref1;  
let ref3 = &mut root;
```

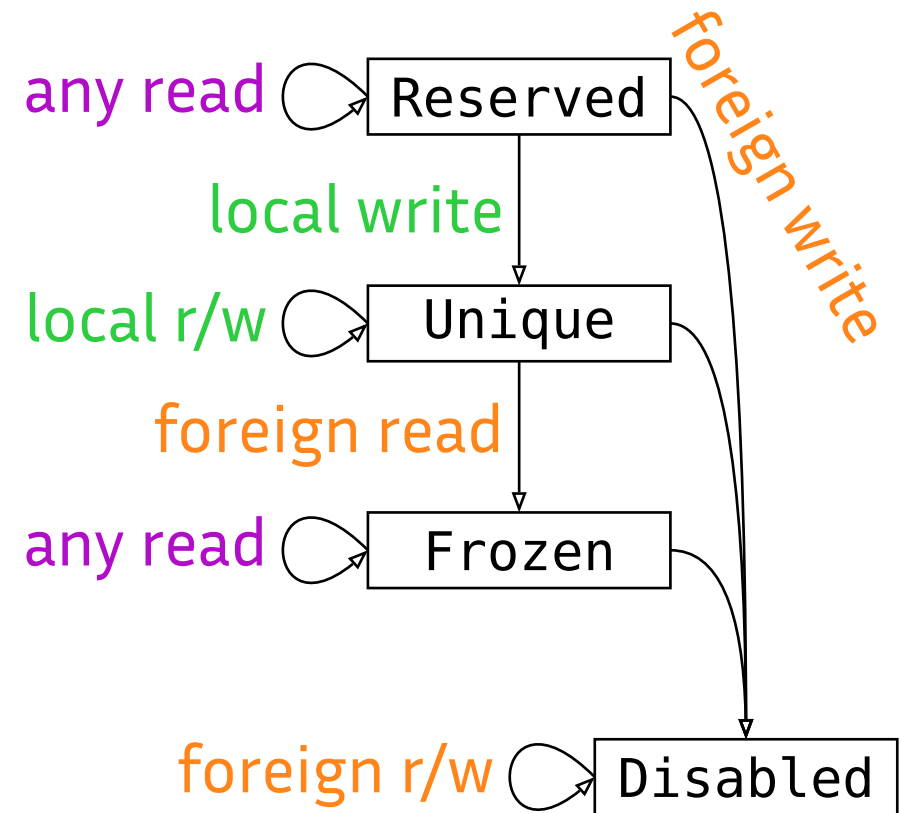
```
let val = *ref2;  
let val = *ref3;  
*ref1 = 36;  
root = 13;
```

foreign accesses

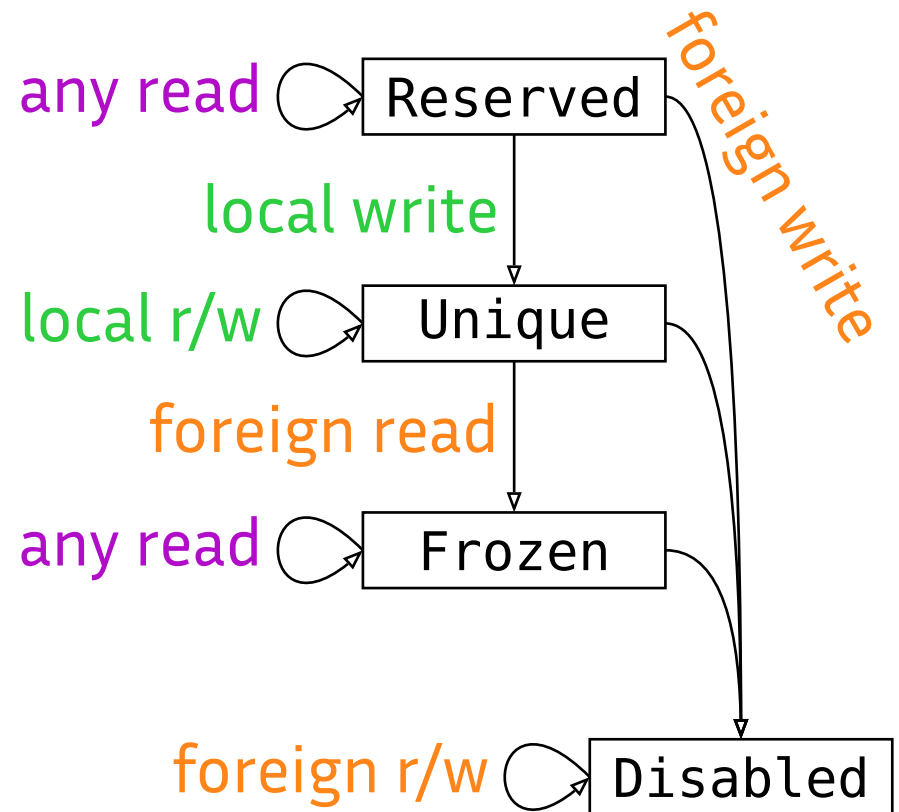


local accesses

Permissions



```
let mut root = 42;  
let x = &mut root;  
let v = *x;  
*x = v + 1;  
let w = root;  
root = 0
```

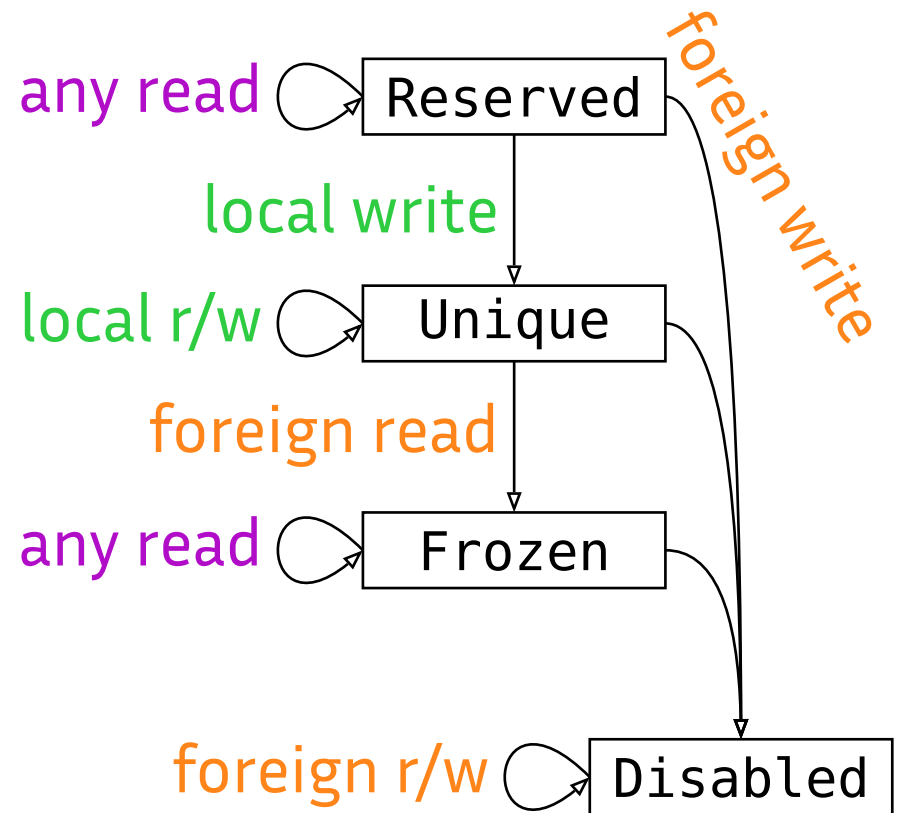
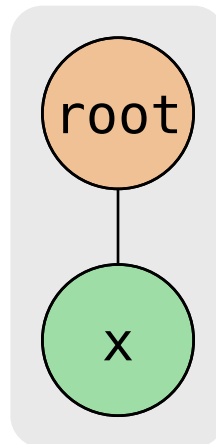


```

let mut root = 42;
let x = &mut root;
let v = *x;
*x = v + 1;
let w = root;
root = 0

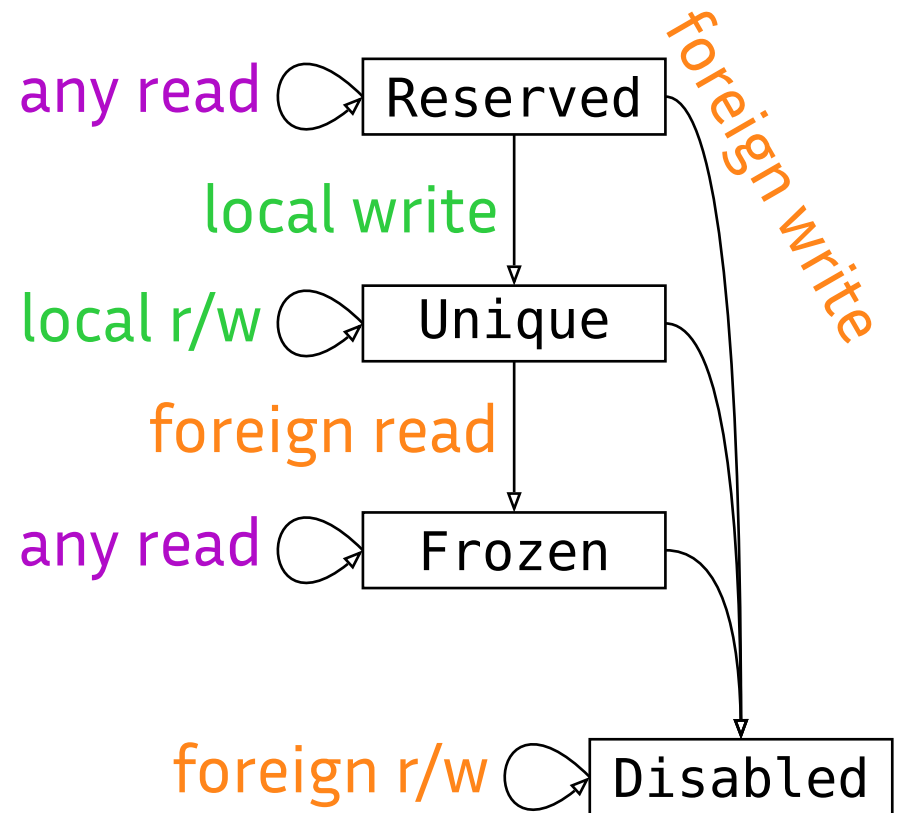
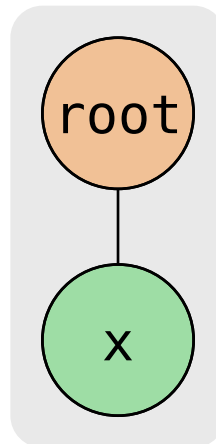
```

x: Reserved (r/w)



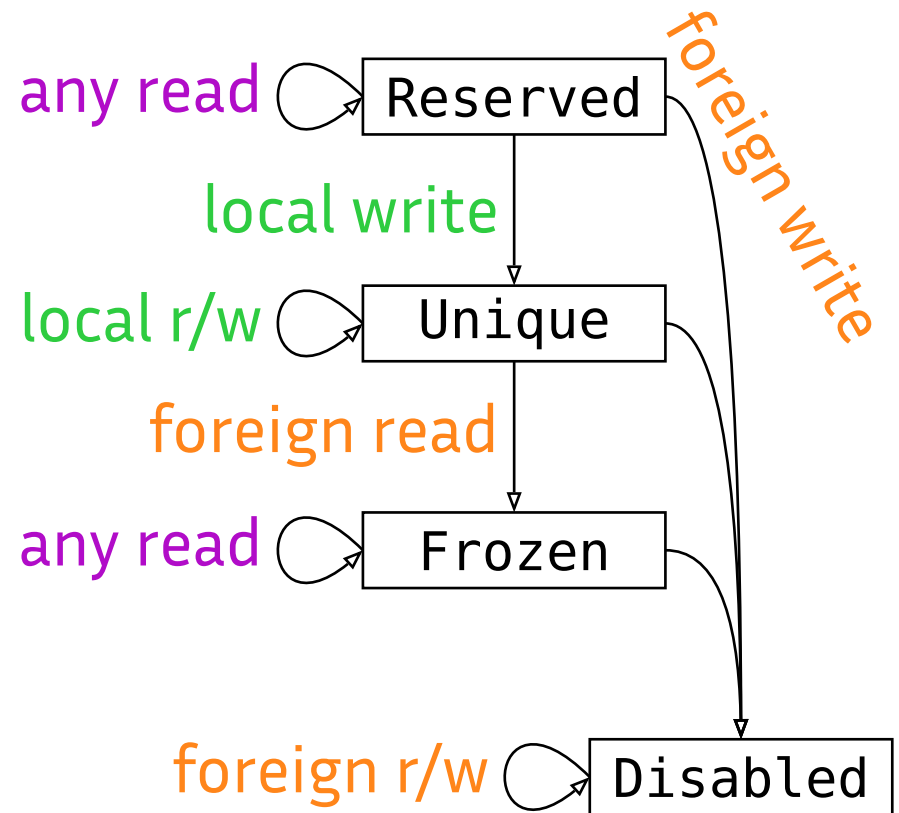
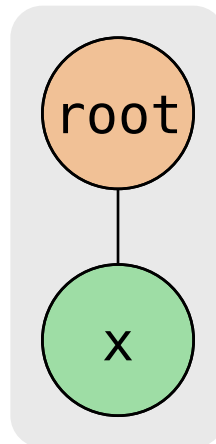
```
let mut root = 42;  
let x = &mut root;  
let v = *x;  
*x = v + 1;  
let w = root;  
root = 0
```

x: Reserved (r/w)



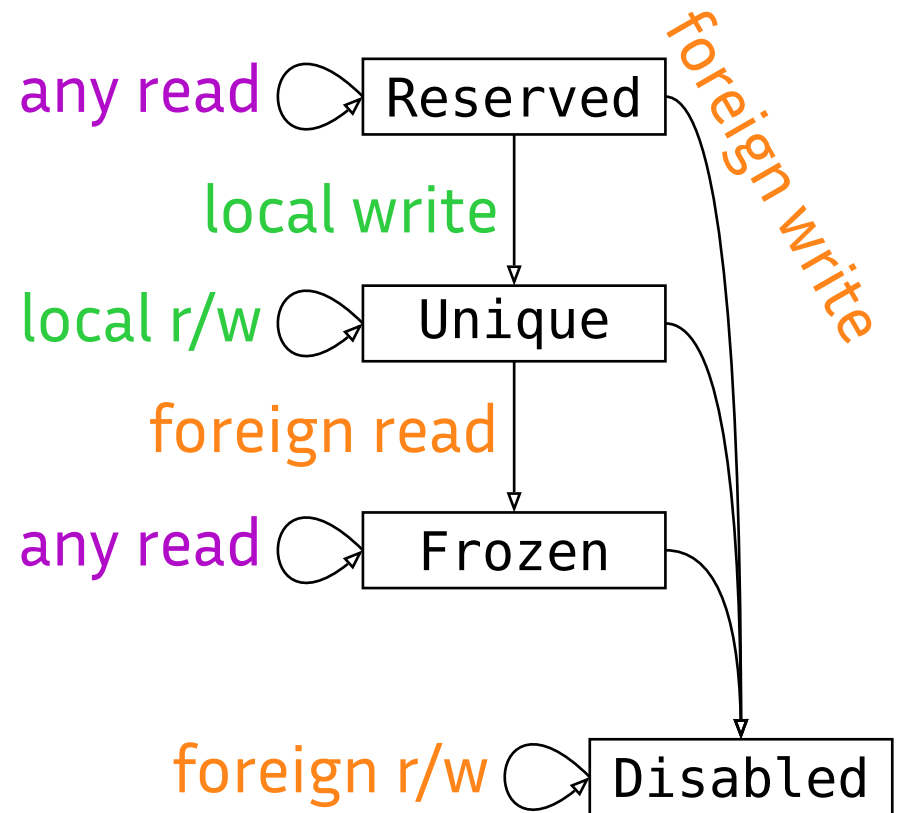
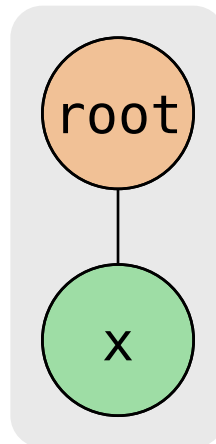
```
let mut root = 42;  
let x = &mut root;  
let v = *x;  
*x = v + 1;  
let w = root;  
root = 0
```

x: Unique (r/w)



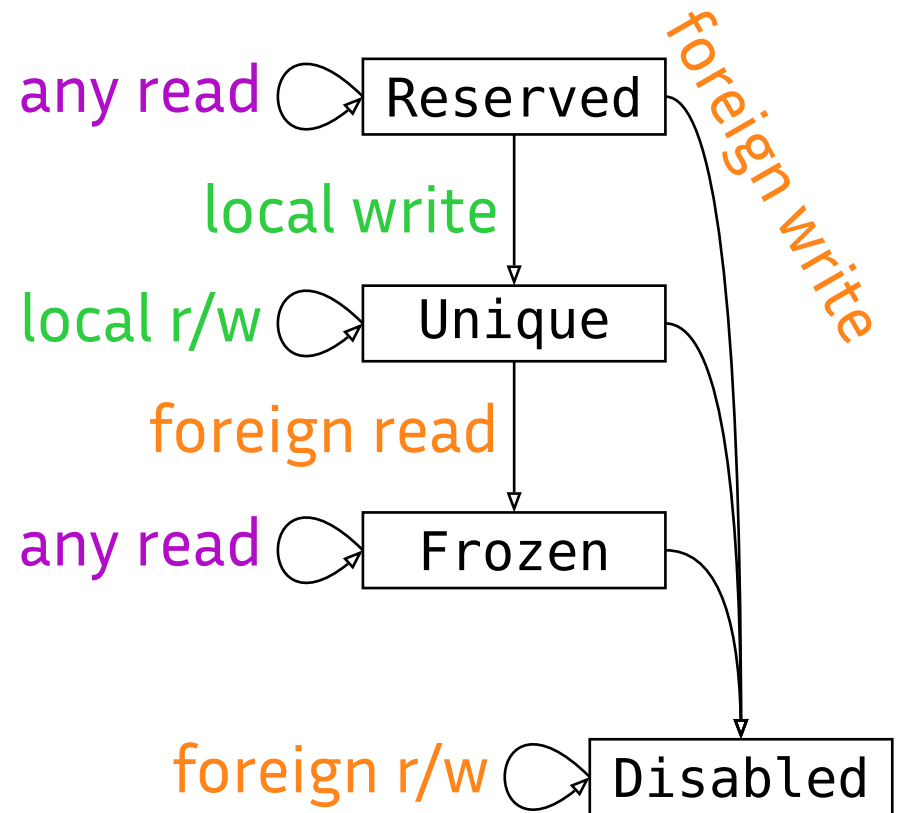
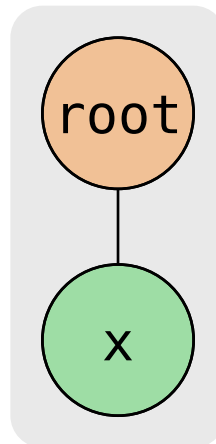
```
let mut root = 42;  
let x = &mut root;  
let v = *x;  
*x = v + 1;  
let w = root;  
root = 0
```

x: Frozen (r)



```
let mut root = 42;  
let x = &mut root;  
let v = *x;  
*x = v + 1;  
let w = root;  
root = 0
```

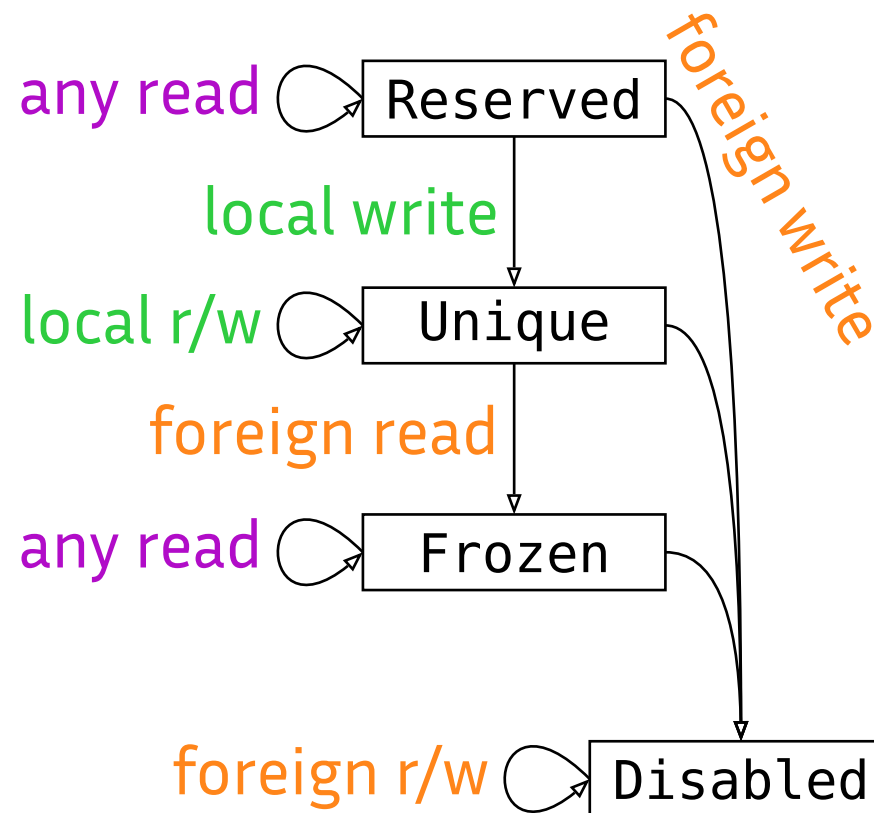
x: Disabled



Addressing Stacked Borrows' limitations

Permissions

- references have static range
→ tree structure
- ignores two-phased borrows
→ Reserved
- prohibits reordering reads
→ Frozen



Evaluation

TB should enable desired optimizations

i.e. have enough UB to rule out problematic patterns

- formalized in Rocq (+Simuliris)
- a selection of optimizations proven
 - ✓ delete read through `&mut` or `&`
 - ✓ insert read through `&` in function
 - ✓ move read down for `&mut` or `&` in function
 - ✓ move write up for Unique `&mut` in function

...

It should be possible to write **unsafe** code free of UB Evaluation

i.e. UB should be predictable and not too common

- implemented in Miri
- tested against 30 000 most downloaded crates on `crates.io`
 - 400 000+ working tests
 - measure how many have UB from Stacked / Tree Borrows

It should be possible to write **unsafe** code free of UB Evaluation

i.e. UB should be predictable and not too common

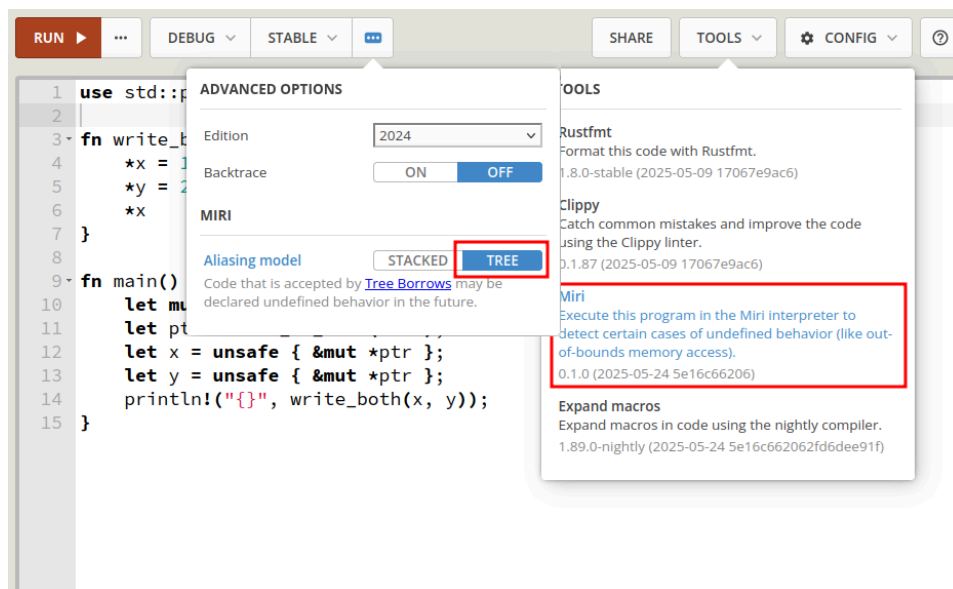
- implemented in Miri
- tested against 30 000 most downloaded crates on `crates.io`
 - 400 000+ working tests
 - measure how many have UB from Stacked / Tree Borrows

Tree Borrows reduces aliasing-related UB by over 50%

Only 31 tests are regressions, all easily fixable.

Conclusion

Try it out: play.rust-lang.org



Learn more:

plf.inf.ethz.ch/research/pldi25-tree-borrows.html

- dynamic ranges
- raw pointers
- interior mutability

